SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 – 50, Q2 – 50
GRAND TOTAL:			**100 Marks**

SECTION A – CONCEPT MAPPING

Note: Reference/sample answers below are provided for five representative questions (Q1–Q5) from the CO1_AT2 concept-mapping question bank.

CO1 AT2 - Concept Mapping Answers

CO1_AT2 – Concept Mapping Answers (Sample: Q1–Q5)

Course: Design and Analysis of Algorithms (CSA06) **CO1:** Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

Q1. Concept Map — Algorithm → Time Complexity → Asymptotic Notation → {Big-O, Big-Theta, Big-Omega}

Problem Definition

The task is to map how an abstract algorithm is transformed, step by step, into a measurable efficiency metric. Starting from the algorithm itself, the map must trace the path through time complexity and asymptotic notation, ending in the three standard bounds (Big-O, Big-Theta, Big-Omega), and must also show how Best-case, Average-case, and Worst-case analysis fit into this structure as children of Time Complexity.

Concept Map Structure

Algorithm

- → Time Complexity (how running time grows with input size n)
- → Best-case Analysis (minimum running time over all inputs of size n)
- → Average-case Analysis (expected running time over a distribution of inputs)
- → Worst-case Analysis (maximum running time over all inputs of size n)
- → Asymptotic Notation (mathematical language used to express each of the above cases)
- → Big-O (O) — upper bound on growth rate
- → Big-Theta (Θ) — tight bound on growth rate
- → Big-Omega (Ω) — lower bound on growth rate

Explaining the Flow (Algorithm → Measurable Metric)

An algorithm by itself is only a sequence of steps; it has no inherent "number" attached to it. The map's first link, Algorithm → Time Complexity, is created the moment we start counting how the number of basic operations grows as input size n increases. This converts a qualitative procedure into a quantitative function of n.

Because the number of operations can differ across the many possible inputs of the same size, Time Complexity itself branches into Best-case, Average-case, and Worst-case — each is a separate function describing a different scenario, not three different algorithms.

The next link, Time Complexity → Asymptotic Notation, is what makes these functions comparable and usable in practice: instead of expressing exact operation counts (which depend on hardware, language, and compiler), we classify the growth pattern using Big-O, Big-Theta, and Big-Omega. This is the step that converts a mathematical function into a standardized, hardware-independent efficiency metric.

How Best/Average/Worst-case Connect to the Notations

Case	Typical Notation Used	What It Represents
Best-case	Ω (Big-Omega)	Lower bound — the fastest the algorithm can run
Average-case	Θ (Big-Theta)	Tight bound — expected behaviour over typical inputs
Worst-case	O (Big-O)	Upper bound — the slowest the algorithm can

		run, used for guarantees

Justification — Why These Branches Together Support Performance Evaluation

- Worst-case (→ Big-O) gives a safety guarantee: the algorithm will never take longer than this, which is essential for real-time or safety-critical systems.
- Best-case (→ Big-Omega) shows the most optimistic scenario, useful for understanding an algorithm's minimum possible cost (e.g., an already-sorted array for insertion sort).
- Average-case (→ Big-Theta, where tight bounds exist) gives the most realistic, practical expectation of performance for typical/random inputs, which is often what matters most in production systems.

Relying on only one case would give an incomplete picture: worst-case alone can make an algorithm look worse than it usually performs, while best-case alone can be misleadingly optimistic. Mapping all three cases onto the three notations gives a complete, three-dimensional view of efficiency — a guaranteed ceiling, a realistic expectation, and an optimistic floor — which together let engineers make informed algorithm-selection decisions.

Q2. Concept Map — Divide & Conquer → Recurrence Relation → Master Theorem → Time Complexity

Problem Definition

This map traces how a design strategy (Divide & Conquer) becomes a piece of mathematics (a recurrence relation), how that mathematics is solved using a standard tool (the Master Theorem), and how the three cases of the Master Theorem branch out of the general recurrence form T(n) = aT(n/b) + f(n).

Concept Map Structure

Divide & Conquer

- → Recurrence Relation: T(n) = aT(n/b) + f(n)
- a = number of subproblems, b = factor by which size shrinks, f(n) = cost of divide + combine
- → Master Theorem (compares f(n) with n^(log_b a))
- → Case 1: f(n) = O(n^(log_b a − ε)) → T(n) = Θ(n^(log_b a))
- → Case 2: f(n) = Θ(n^(log_b a)) → T(n) = Θ(n^(log_b a) · log n)
- → Case 3: f(n) = Ω(n^(log_b a + ε)), regularity holds → T(n) = Θ(f(n))
- → Time Complexity (final closed-form result, one of the three above)

How Each Node Feeds Into the Next

A Divide & Conquer algorithm always follows the same pattern: split the problem into a subproblems each of size n/b, solve them recursively, then combine the results at cost f(n). Writing this pattern down mathematically produces exactly one recurrence relation, T(n) = aT(n/b) + f(n) — this is the Divide & Conquer → Recurrence Relation link.

The Recurrence Relation → Master Theorem link exists because solving a recurrence by repeated substitution or recursion trees is tedious and error-prone; the Master Theorem gives a direct formula by comparing the combine cost f(n) against the "watershed" function n^(log_b a), which represents the total cost contributed by the subproblem-splitting alone.

Sub-links: How the Three Cases Branch Out

Condition (comparing f(n) to n^(log_b a))	Interpretation	Resulting Complexity
f(n) grows polynomially slower	The recursive splitting dominates	Θ(n^(log_b a))

	the cost	
f(n) grows at the same rate	Split-cost and combine-cost are balanced	Θ(n^(log_b a) · log n)
f(n) grows polynomially faster (+ regularity)	The combine step dominates the cost	Θ(f(n))

Justification — How the Map Simplifies Complexity Analysis

- It replaces manual recursion-tree expansion with a single comparison test (f(n) vs n^(log_b a)), turning a calculus-like derivation into a lookup exercise.
- It generalises across all Divide & Conquer algorithms — Merge Sort, Binary Search, Strassen's Matrix Multiplication, etc. — because they all reduce to the same recurrence template.
- The branching into exactly three cases mirrors the only three possible relationships two growth functions can have (slower, equal, faster), so the map is provably exhaustive — every valid recurrence of this form falls into one of the three cases (or is inconclusive, requiring the Akra–Bazzi generalisation).

Overall, the map shows a clean pipeline: an algorithmic design choice (Divide & Conquer) is mechanically converted into a mathematical object (the recurrence), which is then mechanically converted into an efficiency classification (Time Complexity) via a fixed, three-way decision rule (the Master Theorem) — removing the need to re-derive complexity from first principles for every new recursive algorithm.

Q3. Concept Map — Recurrence Relation → Substitution Method → Expansion → Complexity Derivation

Problem Definition

This map traces the full path from a raw recurrence relation to a proven closed-form complexity using the substitution method. The extended map adds Guess → Induction → Verification as intermediate nodes, showing that solving a recurrence is not just algebra but a structured proof process.

Concept Map Structure

Recurrence Relation

- → Substitution Method (technique used when a recurrence does not fit a standard template)
- → Expansion (unrolling the recurrence T(n-1), T(n-2), … repeatedly to expose a pattern)
- → Guess (propose a closed-form candidate, e.g. T(n) = O(n²), based on the expansion pattern)
- → Induction (assume the guess holds for all sizes smaller than n)
- → Verification (substitute the assumption back into the recurrence and check the inequality closes)
- → Complexity Derivation (the final, proven Θ-bound)

How Each Connection Contributes

The Recurrence Relation → Substitution Method link exists because not every recurrence matches the a·T(n/b)+f(n) template the Master Theorem needs — recurrences like T(n) = T(n-1) + n require a more manual, general-purpose technique.

Substitution Method → Expansion is the mechanical first step: writing out several levels of the recurrence by hand (T(n) = T(n-1)+n, T(n-1) = T(n-2)+(n-1), …) until a summation pattern becomes visible.

Expansion → Guess is the creative leap: once the pattern (a sum of the first n integers, for example) is visible, it is converted into a candidate closed form, such as O(n²).

Guess → Induction → Verification is the rigor step: the guess is only accepted once it survives an inductive proof — assume T(k) ≤ ck² for all k < n, substitute into the recurrence, and check the resulting inequality holds for a suitable constant c.

Worked Illustration — T(n) = T(n-1) + n

Stage	What Happens
Expansion	$T(n) = T(n-1)+n = T(n-2)+(n-1)+n = \ldots = T(1) + (2+3+\ldots+n)$
Guess	Sum of first n integers is quadratic → guess $T(n) = O(n^2)$
Induction Hypothesis	Assume $T(k) \leq ck^2$ for all $k < n$
Verification	$T(n) = T(n-1)+n \leq c(n-1)^2+n = cn^2-2cn+c+n \leq cn^2$ for large c, n

Justification / Interpretation

- Without Expansion, the Guess step would be blind trial-and-error rather than a pattern-informed estimate.
- Without Induction and Verification, a plausible-looking guess (e.g. an off-by-one-degree error) could be accepted incorrectly — the proof step is what turns an estimate into a certainty.
- This map generalises beyond one example: any recurrence that decrements by a constant, or splits unevenly, can be solved by the same five-node pipeline.

Overall, the map shows that determining algorithm efficiency by substitution is a disciplined loop of expand → guess → prove, ending only when verification confirms the derived complexity, here $\Theta(n^2)$.

Q4. Concept Map — Algorithm → Input Size (n) → Growth Rate → Efficiency

Problem Definition

This map explains how efficiency is not a fixed property of an algorithm but emerges from the relationship between input size and the rate at which resource use grows. The extended map adds Asymptotic Notations and Scalability as supporting nodes to connect abstract growth patterns to practical usability.

Concept Map Structure

Algorithm

- → Input Size (n) (the parameter that all cost functions are expressed in terms of)
- → Growth Rate (how operation count changes as n increases)
- → Asymptotic Notations (Big-O / Big-Theta / Big-Omega, the formal language for expressing growth rate)
- → Efficiency (the practical judgement of whether the algorithm is usable for the expected n)
- → Scalability (whether efficiency is preserved as n grows toward large, real-world values)

How Each Node Influences the Next

An Algorithm only becomes measurable once Input Size (n) is introduced as a variable — without n, there is nothing to grow. As n increases, the number of basic operations performed traces out the Growth Rate: linear, logarithmic, quadratic, exponential, and so on.

Growth Rate → Asymptotic Notations is the formalisation step: instead of describing growth informally, it is classified using O, Θ, and Ω, which strip away hardware-specific constants and focus purely on the shape of the curve.

Asymptotic Notations → Efficiency is the interpretive step: a classification like $O(n^2)$ is translated into a practical judgement — "acceptable for small n, risky for large n."

Why Asymptotic Notations and Scalability Matter as Supporting Nodes

- Asymptotic Notations remove the noise of machine speed and implementation details, allowing two algorithms to be compared purely on growth pattern.

- Scalability is the forward-looking consequence of growth rate: an algorithm with a low-order growth rate (log n, n) remains efficient even as datasets scale to millions of records, while a high-order growth rate (n^2, 2^n) becomes impractical quickly.
- Linking Growth Rate to both Efficiency and Scalability shows that efficiency is really a two-part judgement: how it performs now, and how it will continue to perform as n grows.

Justification / Interpretation

The overall structure captures a cause-and-effect chain: input size drives growth rate, growth rate is captured formally through asymptotic notation, and that notation is what ultimately determines whether an algorithm is efficient and scalable enough for real-world use. This is why algorithm designers evaluate correctness and efficiency separately — an algorithm can be perfectly correct yet still unusable at scale if its growth rate is too steep.

Q5. Concept Map — Sorting Algorithm → Merge Sort → Recurrence T(n) = 2T(n/2) + O(n) → O(n log n)

Problem Definition

This map traces the conceptual flow from the general category of sorting algorithms down to the specific, provable complexity of Merge Sort, and explains — through side-nodes for its three phases — why that complexity gives Merge Sort an advantage over quadratic sorts.

Concept Map Structure

Sorting Algorithm

- → Merge Sort (a specific divide-and-conquer instance of the sorting category)
- → Divide (split the array into two halves of size n/2)
- → Conquer (recursively sort each half — this is the 2T(n/2) term)
- → Combine (merge the two sorted halves — this is the O(n) term)
- → Recurrence: T(n) = 2T(n/2) + O(n)
- → O(n log n) (final complexity, by the Master Theorem, Case 2)

Mapping the Conceptual Flow

Sorting Algorithm is the parent category; Merge Sort is one member of it, distinguished by its divide-and-conquer strategy. The three phases — Divide, Conquer, Combine — are not just descriptive labels; each maps directly onto a term in the recurrence: Divide contributes the problem-halving structure (the n/2), Conquer contributes the two recursive calls (the 2T(·) term), and Combine contributes the linear-time merge step (the O(n) term).

Once the phases are expressed as the recurrence T(n) = 2T(n/2) + O(n), the Master Theorem is applied: a = 2, b = 2, f(n) = O(n), so n^(log 2) = n, matching f(n) exactly — Case 2 — giving T(n) = Θ(n log n).

Why This Beats O(n^2) Sorts

Aspect	Merge Sort (n log n)	Simple Sort e.g. Bubble/Insertion (n^2)
Structure	Recursively halves the problem each level	Compares/shifts one element at a time
Levels of work	log n levels, O(n) work each → n log n total	n passes, O(n) work each → n^2 total
Scaling to n = 1,000,000	~19.9 million operations	~1,000,000,000,000 operations

Justification / Interpretation

- The Divide/Conquer/Combine side-nodes make the recurrence self-explanatory: each phase's cost is visible as a distinct term instead of an unexplained formula.
- Because the recurrence is balanced (subproblem cost equals combine cost, Master Theorem Case 2), no single phase dominates — the total cost spreads evenly across log n levels, each costing O(n), which is the structural reason n log n beats n^2.

Overall, the map shows that Merge Sort's efficiency advantage is not accidental: it is a direct, traceable consequence of how the Divide, Conquer, and Combine phases translate into a balanced recurrence that the Master Theorem resolves to O(n log n).